

Pact / Contract Testing

Consumer-driven contract testing to reduce integration failures between services.

What Contract Testing is

Contract Testing verifies the agreement between a consumer and a provider without requiring the entire integrated environment to be available. Instead of validating only isolated endpoints, it checks whether requests and responses still respect the expectations that one service has about another. This is especially useful in distributed systems and microservices, where interface changes can break consumers independently of the provider implementation.

How Pact works

Pact records consumer expectations as contracts (pacts). The consumer test defines the interaction it needs; the provider later verifies those interactions against its implementation. This creates fast feedback when an API change would violate an agreed request, response, status, header or payload structure.

Consumer and Provider

The Consumer is the application or service that uses an API. The Provider exposes the API. A good contract focuses on what the consumer really depends on, avoiding unnecessary fields that make the contract brittle. QA can help identify critical interactions and negative or compatibility scenarios.

Pact Broker/PactFlow

A Pact Broker or PactFlow can publish, version and share contracts and verification results across teams and pipelines. This enables visibility of which consumer/provider versions are compatible and supports deployment decisions based on verified contracts.

QA value

QA adds value by identifying risky integrations, reviewing meaningful consumer expectations, validating error contracts and ensuring that contract verification runs in CI/CD. Contract tests complement, rather than replace, integration and end-to-end testing.

Attention point

Do not turn a contract into a copy of the entire provider response. Overly strict contracts create false failures and excessive maintenance. Model only behavior the consumer truly requires and keep versioning and verification results traceable.